

11-03-2026

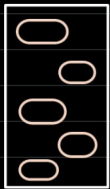
Agenda:

Database - I

python, numpy, pandas, data visualization → Data science → AI

Database → manage product/project data → software

whatsapp ↔ persist ↔ data ↔ system



→ software → create → C
→ update → U → CRUD
→ Read → R
→ Delete → D
+ more manage

python → problem statement → University
→ students data → store
→ update
→ Delete
→ Read → id

python → file system → os
→ open()
↓
file operations → r
→ w
→ a+
→ wt

store → add new entry
→ open("data.txt", "a+")

.txt 2 2

101, Menal, @gmail.com, 10th, 01-10-2006
102, Nile, @gmail.com, 11th, _____
103, -, -, -, -, _____

103 → readline() → [] → 0 → line 1
1 → line 2

func update (id, email):
← 103 → g@gmail.com

with open (____.txt, r) as f:

all [] = f.readline()

new-entry = []

for s in all:

if s[0] == id:

s[2] = email

new-entry.append(s)

else:

new-entry.append(s)

data, text. write all (new-entry)

problem with current approach:

PS. 1

store → company data → employees.csv

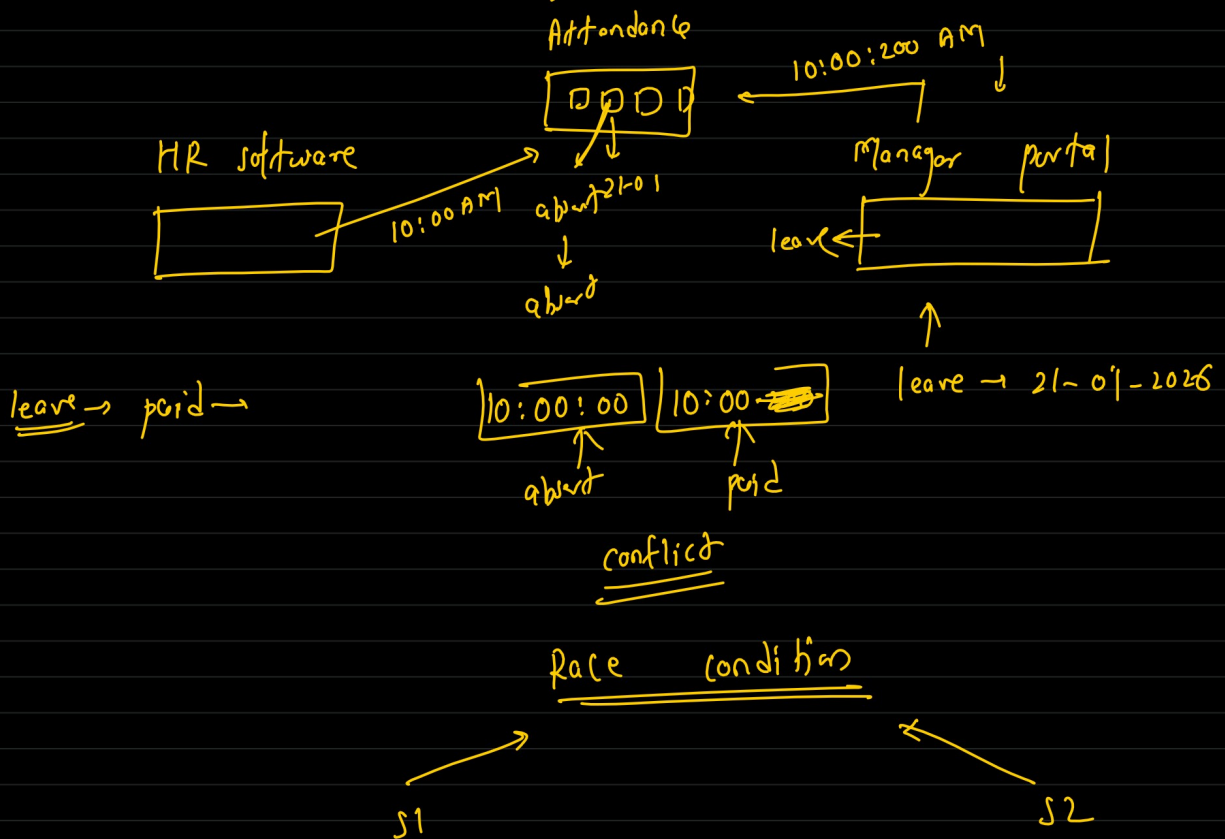
↑
file system



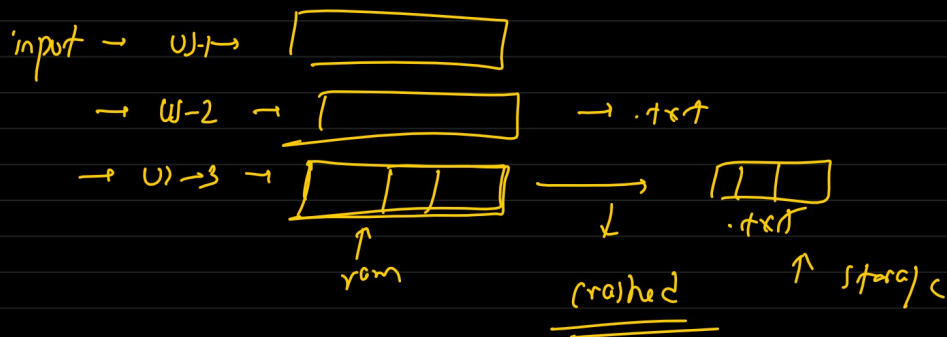
employee count → 5,00,000 lakh

→ task → employee id = 3 → fast → searching becomes
→ employee id = 4,99,999 → slow

PS. 2 Multiple program accessing data.



PS. 3 Data Corruption



P. 1

program → process → OOP → manage logic, provide packing & modules

→ to define standard using ABC

→ Data security

→ to merge our data with the logic

Real world, data is connected,

Relationship Between Data

Employee

id	Name
1	—
2	—
3	—

Department

dept-id	dept-name
1	IT
2	HR
3	Business



IT → IT_A

Emp-department

emp-id	dept-id
1	1
2	1
3	2

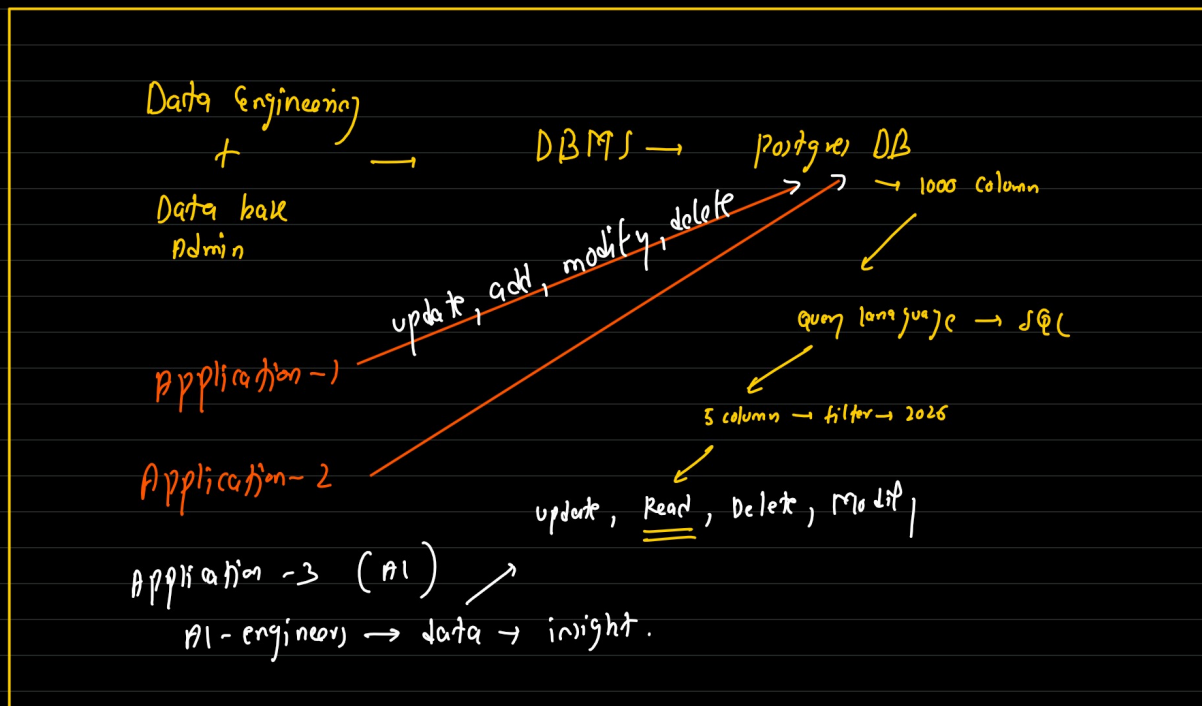
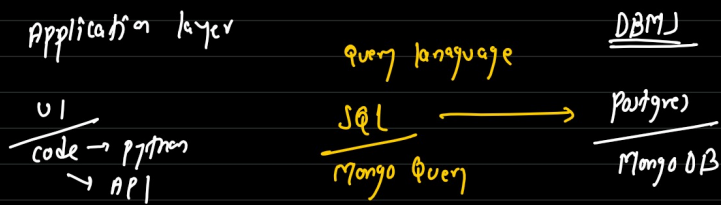
- p. searching becomes slow
- p. Race condition (Multiple program Accessing Data)
- p. Data corruption
- p. Relationship between data.

DBMS → Database Management System

- (software) → postgres
- Mongo DB
- mysql

DB → collection data

Structure of Database System



Types of Database

Relational Database

stores data in tables (rows & columns)
that have relationship with each other.

Employee Table

id	name	dept	salary
1	rahul	IT	70k
2	Neha	HR	60k

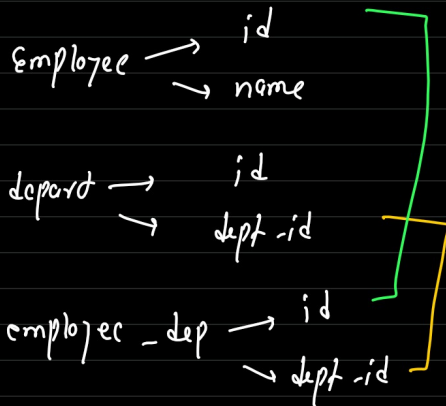
Non-relational Database

NoSQL stores data in flexible formats instead of rigid tables

Relational DB



tables are connected using relationship



This relationship is maintained using keys



relational_data_table (2024) → .csv

post-id user timestamp caption like

→ insert →

(2025) ↙

post-id user timestamp caption like comments shared

||||| → flexibility

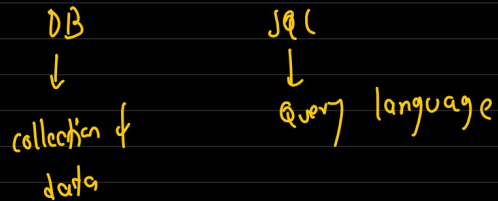
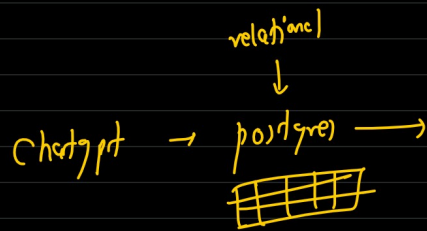
dict

dict
post-id:
user:
timestamp
caption:
like

dict / json

dict / json
post-id:
user:
timestamp
caption:
like:
comment: {
user:
likes: 4, 1, 1, 6
}

q[" "] = value



System design:

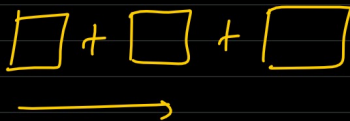
Horizontal scaling

2030 (32gb ram)

System → i3 cpu
→ 12gb ram
→ 256gb r

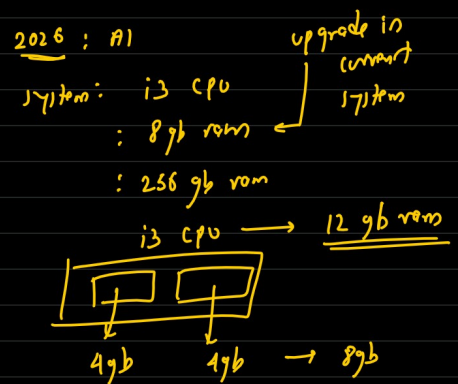
+

System-2 → i5 cpu
→ 20gb ram
→

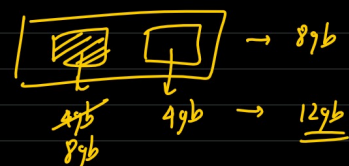


Vertical scaling

2018
System → i3 cpu
→ 4gb ram
→ 256gb ram



2028 : cpu → i3
→ 12gb
→ 256gb



relational DB

non-relational DB

vertical scaling



horizontal scaling

Core Structure of Relational Database

1) Table

Structure collection of data organized into rows & columns.

employee →
id integer
name string
department string
salary integer

SQL code

create the table:

CREATE TABLE employees (

id INT
name TEXT
department TEXT ← VARCHAR
salary INT
,

SQL line

);

2) Column

Syntax: column-name, data-type, constraint)



→ name, VARCHAR, NOT NULL

SQL-like → Basic Data Type ← postgres

NULL, INT, REAL,
TEXT, BLOB

CHAR, VARCHAR, TEXT
DATE, TIME, TIMESTAMP,

3) Row (Record)

represents single record in a table.

SQL Query:

insert data in employees table:

```
CREATE TABLE employees (
```

```
  id INT,  
  name TEXT NOT NULL,  
  department TEXT,  
  salary INT ,
```

```
);
```

→ insert → employees

```
INSERT INTO employees VALUES (1, 'Rahul', 'IT', 70000);
```

```
INSERT INTO employees VALUES (1, None, 'IT', 70000); → error
```

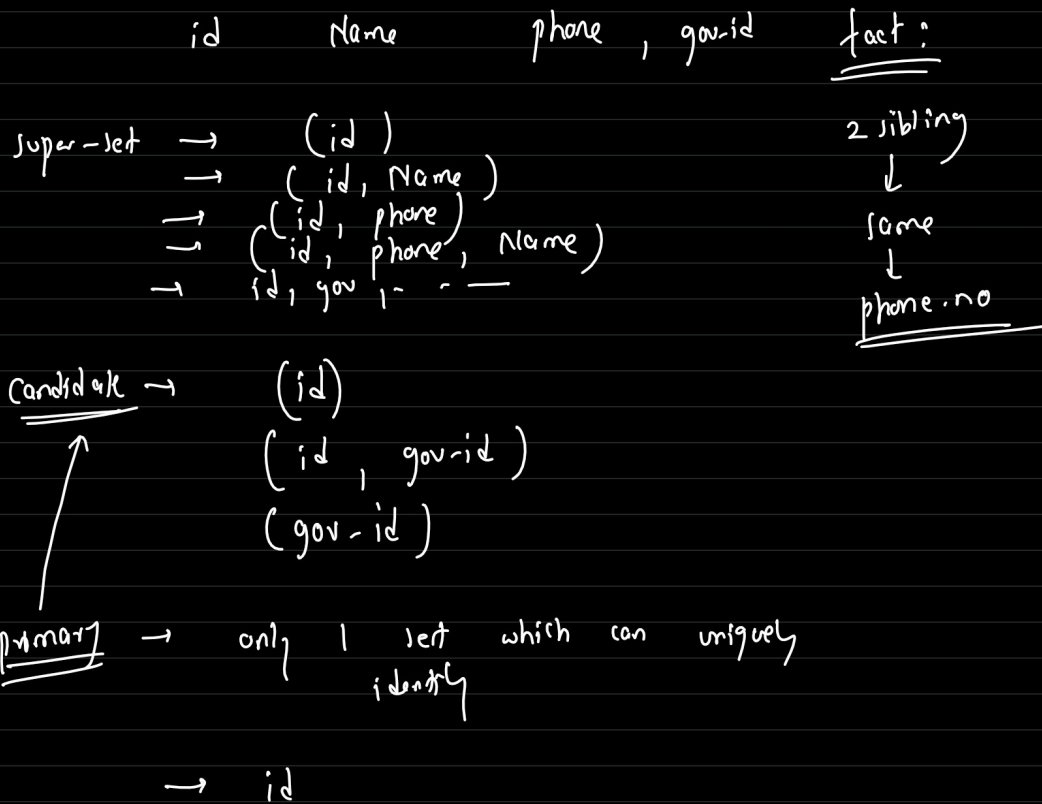
```
INSERT INTO employees VALUES (1, 'Rahul', 'IT', 70000),
```

```
(2, 'Rahul', 'IT', 70000),
```

```
(3, 'Rahul', 'IT', 70000);
```

↓ phen

Rahul	98000 0000 ←
Rahul	98000 0000 ←



```
CREATE TABLE employees (
  id INT PRIMARY KEY,
  name TEXT NOT NULL,
  department TEXT,
  salary INT,
);
```